

Progetto “Negozzi”

Cordo Basi di Dati – Unimi (aa 2024-25)

Autore: Alessandro Valenti

Matricola: 41913A

Corso: Basi di Dati — Laboratorio - Progetto 2025

Introduzione

Obiettivo: sviluppare un'applicazione per la gestione di una catena di negozi, con funzionalità per clienti e manager. I clienti possono accedere tramite login al negozio online che presenta la lista dei prodotti che possono essere acquistati scegliendo per ogni prodotto da quale negozio acquistare e la quantità. Allo stesso modo il manager accede tramite login e accede a una dashboard di gestione: negozi, clienti, prodotti, fornitori.

Analisi dei requisiti

1. Scopo e contesto

Realizzare un'applicazione per la gestione di una **catena di negozi**, con funzionalità per **clienti** e **manager**: i clienti consultano i prodotti, li acquistano e possono applicare sconti su fattura grazie ai punti accumulati nella tessera fedeltà; le tessere sono rilasciate dai negozi ma sono valide per acquisti nel sito e quindi in tutti i negozi; i manager gestiscono anagrafiche e processi (prodotti, prezzi per negozio, negozi, clienti, fornitori, ordini ai fornitori, carte fedeltà). Per comodità su richiesta progettuale è stato creato un solo manager.

2. Stakeholder e attori

- Cliente (utente finale): i clienti (dati utente e dati personali) sono aggiunti dal manager. Il cliente quindi accede con credenziali iniziali, la username corrisponde alla email e una password default. Il cliente può cambiare la password in qualsiasi momento, una volta entrato nel sito con le credenziali visualizza prodotti, effettua acquisti, può applicare uno sconto disponibile a seconda dei punti accumulati per un massimo di 100 euro, e vedere il saldo punti della propria tessera. Solo il manager può eliminare il cliente.

- Manager (gestore catena): accede con credenziali (il sistema nasce con le credenziali di amministratore), può cambiare password; crea e gestisce utenze clienti; inserisce e gestisce i prodotti (con prezzi differenziati per negozio), gestisce i negozi e i fornitori; inserisce ordini ai fornitori tramite il magazzino/listino negozio per rifornimenti a uno specifico negozio per uno specifico prodotto. L'ordine viene indirizzato automaticamente al negozio che per quel prodotto garantisce il prezzo migliore.

2.1 Altre Entità

- Negozi: ogni punto vendita è una entità autonoma nella catena e viene identificato da un codice, ogni negozio ha un responsabile, orari di apertura (gestiti separatamente) e indirizzo. È il negozio che rilascia la tessera fedeltà al cliente, sempre a gestione del manager, dalla lista dei propri tesserati, scegliendo tra i clienti che non possiedono ancora una tessera fedeltà. Se non esistono clienti senza tessera la funzione viene disabilitata.

- Fornitori: sono i punti di rifornimento dei prodotti. Sono identificati da un numero di PIVA (11 caratteri numerici che possono cominciare con 0). Ogni fornitore fornisce più prodotti e ogni prodotto può essere venduto da più fornitori. Sono coinvolti negli *ordini* e alla fine di ogni ordine al fornitore viene aggiornata la quantità a disposizione (diminuita della quantità richiesta), l'ordine riceve una data d'ordine e una presunta data di arrivo dell'ordine (viene generata automaticamente 7gg dalla data d'ordine), il negozio vede la lista di ordini in attesa e può segnalarli come arrivati, a quel punto la quantità ordinata aumenta il magazzino del negozio. I fornitori possono controllare la lista (storica) dei propri ordini con lo stato (emesso, consegnato, annullato).

- Fatture le fatture hanno un numero unico progressivo (unico per la catena di negozi), il nome del cliente, una data, uno sconto percentuale applicato grazie ai punti del cliente (deve essere esplicitamente chiesto dal cliente) e il totale pagato (lordo meno lo sconto). Il dettaglio delle fatture (prodotto, negozio, quantità, prezzo) vengono salvati separatamente. Il prezzo lordo pagato lo ricaviamo dall'ordine (quantità*prezzo). La fattura, mantenendo lo sconto percentuale, aggiorna il prezzo pagato (al netto dello sconto).

- Ordini sono eseguiti dai negozi ai fornitori per aumentare il proprio magazzino. Un ordine ha una propria id, è associato a un negozio da un fornitore per un prodotto, con una certa quantità richiesta. L'ordine tiene traccia della data di effettuazione dell'ordine della presunta data di consegna (per default 7gg) e lo stato dell'ordine. All'inserimento dell'ordine la quantità del prodotto nel magazzino del fornitore diminuisce e l'ordine ha uno stato di "emesso". Il negozio può flaggare l'ordine come "consegnato", a quel punto la quantità aumenta nel suo listino, oppure annullato, e la quantità viene riaccreditata al magazzino del fornitore.

3. Tabelle

```
CREATE TABLE auth.utenti (  
  id_utente serial NOT NULL,  
  ruolo integer NOT NULL,  
  email text NOT NULL,  
  password text NOT NULL,  
  attivo boolean NOT NULL DEFAULT FALSE,  
  creato timestamp NOT NULL DEFAULT now(),  
  ultimo_accesso timestamp,  
  CONSTRAINT id_utente_pk PRIMARY KEY (id_utente),  
  CONSTRAINT unique_email UNIQUE (email)  
);
```

```
CREATE TABLE auth.ruolo (  
  id_ruolo serial NOT NULL,  
  nome text NOT NULL,  
  descrizione text,  
  CONSTRAINT nome_univoco UNIQUE (nome),  
  CONSTRAINT ruolo_pk PRIMARY KEY (id_ruolo)  
);
```

```
CREATE TABLE negozi.clienti (  
  id_cliente serial NOT NULL,  
  cf char(16) NOT NULL,  
  nome text NOT NULL,  
  cognome text,  
  utente integer NOT NULL,  
  telefono text,  
  tessera integer,  
  CONSTRAINT clienti_pk PRIMARY KEY (id_cliente),  
  CONSTRAINT cf_unique UNIQUE (cf),  
  CONSTRAINT tessera_unica UNIQUE (tessera)  
);
```

```
CREATE TABLE negozi.negozi (  
  id_negozio serial NOT NULL,  
  nome_negozio text NOT NULL,  
  responsabile text NOT NULL,  
  indirizzo text NOT NULL,  
  attivo boolean NOT NULL DEFAULT true,  
  CONSTRAINT negozio_pk PRIMARY KEY (id_negozio)  
);
```

```
CREATE TABLE negozi.tessere (  
  id_tessera serial NOT NULL,  
  negozio_emittente integer NOT NULL,  
  data_richiesta date NOT NULL DEFAULT now(),  
  saldo_punti integer NOT NULL DEFAULT 0,  
  archiviata boolean NOT NULL DEFAULT FALSE,
```

```

CONSTRAINT tessere_pk PRIMARY KEY (id_tessera),
CONSTRAINT saldo_punti_neg CHECK (saldo_punti >= 0)
);

CREATE TABLE negozi.storico_tessere (
id_storico serial NOT NULL,
codice_tessera integer NOT NULL,
cliente integer,
saldo_punti integer,
negozio_emittente integer NOT NULL,
data_emissione date NOT NULL,
CONSTRAINT storico_tessere_pk PRIMARY KEY (id_storico)
);

CREATE TABLE negozi.prodotti (
id_prodotto serial NOT NULL,
nome_prodotto text NOT NULL,
descrizione text,
immagine_url text,
CONSTRAINT prodotti_pk PRIMARY KEY (id_prodotto)
);

CREATE TABLE negozi.orari (
negozio integer NOT NULL,
dow smallint NOT NULL,
iod smallint NOT NULL,
apertura time NOT NULL,
chiusura time NOT NULL,
CONSTRAINT dow_1_7 CHECK (dow BETWEEN 1 AND 7),
CONSTRAINT iod_1_2 CHECK (iod IN (1,2)),
CONSTRAINT orari_pk PRIMARY KEY (negozio,dow,iod)
);

CREATE TABLE negozi.listino_negozio (
negozio integer NOT NULL,
prodotto integer NOT NULL,
prezzo_listino numeric(10,2) NOT NULL,
magazzino integer NOT NULL DEFAULT 0,
CONSTRAINT listino_negozio_pk PRIMARY KEY (negozio,prodotto)
);

CREATE TABLE negozi.fornitori (
piva char(11) NOT NULL,
nome_fornitore text NOT NULL,
indirizzo text,
email text,
telefono text,
attivo boolean NOT NULL DEFAULT true,
CONSTRAINT fornitori_pk PRIMARY KEY (piva)
);

```

```
CREATE TABLE negozi.magazzino_fornitore (  
  piva_fornitore char(11),  
  prodotto integer NOT NULL,  
  quantita integer,  
  prezzo numeric(10,2),  
  CONSTRAINT quantita CHECK (quantita>=0),  
  CONSTRAINT prezzo CHECK (prezzo>=0),  
  CONSTRAINT fornitore_prodotto UNIQUE (piva_fornitore,prodotto)  
);
```

```
CREATE TABLE negozi.ordini_fornitori (  
  id_ordine serial NOT NULL,  
  fornitore char(11) NOT NULL,  
  negozio integer,  
  prodotto integer,  
  quantita integer NOT NULL,  
  data_ordine timestamp NOT NULL DEFAULT now(),  
  data_consegna date NOT NULL,  
  stato_ordine text NOT NULL,  
  CONSTRAINT ordini_fornitori_pk PRIMARY KEY (id_ordine),  
  CONSTRAINT check_stato CHECK (stato_ordine IN  
  ('emesso', 'consegnato', 'annullato'))  
);
```

```
CREATE TABLE negozi.fatture (  
  id_fattura serial NOT NULL,  
  cliente integer NOT NULL,  
  data_fattura date NOT NULL DEFAULT now(),  
  sconto_percentuale numeric(3) NOT NULL DEFAULT 0,  
  totale_pagato numeric(10,2) NOT NULL DEFAULT 0,  
  CONSTRAINT fatture_pk PRIMARY KEY (id_fattura),  
  CONSTRAINT totale_check CHECK (totale_pagato >= 0)  
);
```

```
CREATE TABLE negozi.dettagli_fattura (  
  fattura integer NOT NULL,  
  prodotto integer NOT NULL,  
  quantita integer NOT NULL,  
  prezzo_unita numeric(10,2) NOT NULL,  
  CONSTRAINT dettagli_fattura_pk PRIMARY KEY (fattura,prodotto)  
);
```

```
CREATE TABLE negozi.livelli_sconto (  
  sconto_percentuale smallint NOT NULL,  
  punti_richiesti integer NOT NULL  
);  
INSERT INTO negozi.livelli_sconto (sconto_percentuale,  
  punti_richiesti) VALUES (E'5', E'100');  
INSERT INTO negozi.livelli_sconto (sconto_percentuale,
```

```
punti_richiesti) VALUES (E'15', E'200');  
INSERT INTO negozi.livelli_sconto (sconto_percentuale,  
punti_richiesti) VALUES (E'30', E'300');
```

4. Viste

```
CREATE VIEW negozi.v_saldi_punti_300  
AS  
SELECT c.nome, c.cognome, n.nome_negozio, t.saldo_punti,  
       t.data_richiesta  
FROM clienti c  
LEFT JOIN tessere t ON c.tessera = t.id_tessera  
LEFT JOIN negozi n ON n.id_negozio = t.negozio_emittente  
WHERE saldo_punti > 300;
```

Questo è il tipico caso in cui bisogna valutare il trade-off tra aggiornamenti e query. La scelta di una view normale rispetto a una materializzata è legata al fatto che valuto che i refresh della view materializzata possano essere maggiori (all'aggiornamento dei punti o al superamento dei 300 punti su una tessera tramite un trigger) rispetto alla complessità della view normale (2 join fra tre tabelle clienti, tessere, negozi), introducendo overhead e inconsistenza temporanea, senza benefici significativi in termini di prestazione.

```
CREATE VIEW negozi.v_archivio_tessere  
AS  
SELECT c.nome, c.cognome, st.codice_tessera, n.nome_negozio,  
       st.saldo_punti  
FROM negozi.storico_tessere st  
JOIN negozi.clienti c ON st.cliente = c.id_cliente  
JOIN negozi.negozi n ON st.negozio_emittente = n.id_negozio;
```

Questo è il tipico caso in cui bisogna valutare il trade-off tra aggiornamenti e query. La scelta di una view normale rispetto a una materializzata è legata al fatto che valuto che i refresh della view materializzata possano essere maggiori (all'aggiornamento dei punti o al superamento dei 300 punti su una tessera tramite un trigger) rispetto alla complessità della view normale (2 join fra tre tabelle clienti, tessere, negozi), introducendo overhead e inconsistenza temporanea, senza benefici significativi in termini di prestazione.

5. Logica interna al DB (strutture attive)

5.1 Vincoli di Integrità Strutturali e Semantici

Lo schema relazionale progettato include un insieme articolato di **vincoli di integrità strutturali e semantici**, finalizzati a garantire la correttezza dei dati e il rispetto dei requisiti del dominio applicativo descritti nelle specifiche del progetto. Non includiamo la scelta delle chiavi primarie e dei vincoli NULL.

La tabella `auth.utenti` è vincolata da:

- **vincolo di unicità sull'email** (`unique_email`), che garantisce che ogni credenziale di login sia associata ad un solo utente;

La tabella di associazione `auth.utente_ruolo` utilizza:

- **chiave primaria composta** (`id_utente`, `id_ruolo`), che impedisce l'assegnazione duplicata dello stesso ruolo allo stesso utente e modella correttamente la relazione multi-a-molti tra utenti e ruoli. La scelta, anche se non necessaria da specifiche, è dettata dall'idea di poter creare ruoli diversi (gestione fornitore e negozi) per un singolo utente.

La tabella `negozi.clienti` include:

- **vincolo di unicità sul codice fiscale (cf)**, in accordo con il requisito che ogni cliente sia identificato univocamente;
- **vincolo di unicità sul campo tessera**, che garantisce il requisito "ogni cliente può possedere al più una tessera fedeltà".

La tabella `negozi.tessere` è vincolata da:

- **vincolo di CHECK su saldo_punti ≥ 0** , che impedisce la presenza di saldi punti negativi;

La tabella `negozi.orari` è vincolata da:

- **vincoli di CHECK:**
 - `dow BETWEEN 1 AND 7` per rappresentare correttamente i giorni della settimana;
 - `iod IN (1, 2)` per distinguere eventuali fasce orarie multiple (AM, PM).

La tabella `negozi.listino_negozio` è caratterizzata da:

- **valore di default e vincolo implicito sul magazzino**, che garantisce una gestione coerente delle disponibilità locali.

La tabella `negozi.magazzino_fornitore` include:

- **vincolo di unicità (piva_fornitore, prodotto)**, che impedisce duplicazioni dello stesso prodotto per lo stesso fornitore;

- **vincoli di CHECK** su `quantita >= 0` e `prezzo >= 0`, assicurando la coerenza delle informazioni di disponibilità e costo.

La tabella `negozi.ordini_fornitori` presenta:

- **vincolo di CHECK sullo stato dell'ordine**, che limita i valori ammessi a `'emesso'`, `'consegnato'`, `'annullato'`, garantendo la correttezza semantica del ciclo di vita dell'ordine.

La tabella `negozi.fatture` include:

- **vincolo di CHECK su totale_pagato >= 0**, che impedisce valori economicamente non validi;
- **campo sconto_percentuale con valore di default**, in accordo con la gestione degli sconti basata sui punti fedeltà. Non viene effettuato nessun check con la tabella `livelli_sconto` per non precludere possibilità di cambiare, aggiungere, togliere livelli di sconto. In alternativa potevamo elaborare la logica degli sconti con in fatture ho `sconto_percentuale` altri attributi (`attivo`, `data_fine_promozione` ecc...) collegare la tabella `fatture` su una chiave `id_sconto`.

La tabella `negozi.livelli_sconto` definisce una **corrispondenza fissa tra sconto percentuale e punti richiesti**, permettendo di implementare in modo strutturato il requisito di:

- sblocco sconti a soglie predefinite (100, 200, 300 punti),
- utilizzo controllato dei punti tramite funzioni e trigger.

5.2 Triggers

Nella creazione dei trigger ho valutato i due approcci tipici:

- creazione di un trigger evento in una tabella che chiama una funzione wrapper con `return type TRIGGER` che chiama una funzione con `return type VOID`;
- creazione di un trigger evento in una tabella che chiama la funzione con `return type TRIGGER` ma che esegue la funzione direttamente.

Essendo per la maggior parte funzioni molto semplici e mai chiamate direttamente da codice PHP si poteva scegliere il secondo approccio. Per chiarezza ho creato funzioni del primo tipo tranne nel caso della tabella `fatture`.

```
CREATE TRIGGER trg_aggiorna_punti
  AFTER UPDATE OF totale_pagato
  ON negozi.fatture
  FOR EACH ROW
  EXECUTE PROCEDURE negozi.trg_aggiorna_punti_tessera();
```



```

CREATE FUNCTION negozi.trg_aggiorna_punti_tessera ()
RETURNS trigger
LANGUAGE plpgsql
VOLATILE
CALLED ON NULL INPUT
SECURITY INVOKER
PARALLEL UNSAFE
COST 1
AS $$
DECLARE
    v_punti INT;
    v_tessera INT;
BEGIN

SELECT tessera INTO v_tessera
FROM negozi.clienti
WHERE id_cliente = NEW.cliente;
IF v_tessera IS NULL THEN
RETURN NEW;
END IF;
v_punti := floor(NEW.totale_pagato)::INT;
UPDATE negozi.tessere
SET saldo_punti = saldo_punti + v_punti
WHERE id_tessera = v_tessera;

RETURN NEW;
END;
$$;

```

aggiorna_punti()

quando viene aggiornato *totale_pagato* in un record della tabella *fatture* la funzione ricalcola i punti e aggiorna il totale della tessera del cliente associato alla fattura.

Contesto:

Trigger AFTER UPDATE di *totale_pagato* nella tabella *fatture*

```

CREATE TRIGGER trg_archivia_tessere_negozi
BEFORE DELETE
ON negozi.negozi
FOR EACH ROW
EXECUTE PROCEDURE negozi.trg_archivia_tessere_negozi();

```

```

CREATE FUNCTION negozi.trg_archivia_tessere_negozio ()
RETURNS trigger
LANGUAGE plpgsql
VOLATILE
CALLED ON NULL INPUT
SECURITY INVOKER
PARALLEL UNSAFE
COST 1
AS $$
BEGIN
PERFORM negozi.archivia_tessere_negozio (
OLD.id_negozio
);
RETURN OLD;
END;
$$;

CREATE FUNCTION negozi.archivia_tessere_negozio (_idnegozio
integer)
RETURNS void
LANGUAGE plpgsql
VOLATILE
CALLED ON NULL INPUT
SECURITY INVOKER
PARALLEL UNSAFE
COST 1
AS $$
BEGIN
    INSERT INTO negozi.storico_tessere
(codice_tessera, cliente, saldo_punti, negozio_emittente,
data_emissione)
SELECT t.id_tessera, c.id_cliente, t.saldo_punti,
        t.negozio_emittente, t.data_richiesta
    FROM negozi.tessere t
    left JOIN negozi.clienti c ON c.tessera = t.id_tessera
    WHERE t.negozio_emittente = _idnegozio;
DELETE FROM negozi.tessere
    WHERE negozio_emittente = _idnegozio;

RETURN;
END;
$$;

```

archivia_tessere_negozio(): quando viene cancellato un negozio BEFORE viene chiamata una funzione con il valore OLD che inserisce nella tabella STORICO_TESSERE le tessere emesse da quel negozio e le cancella dalla tabella TESSERE. La scelta progettuale è stata quella di rispettare alla lettera la richiesta che è stata interpretata come un HARD delete dalla tabella e la creazione di una nuova tabella. Per mantenere il vincolo referenziale con clienti.tessere (FK tessere.id_tessera) c'è un constraint che on DELETE (nella tabella tessere) viene settato a NULL il valore cliente.tessera.

Contesto:

Trigger BEFORE DELETE di un record nella tabella negozi

```
CREATE TRIGGER trg_aggiorna_magazzino
AFTER INSERT
ON negozi.ordini_fornitori
FOR EACH ROW
EXECUTE PROCEDURE negozi.trg_aggiorna_magazzino_fornitore();

CREATE FUNCTION negozi.trg_aggiorna_magazzino_fornitore ()
RETURNS trigger
LANGUAGE plpgsql
VOLATILE
CALLED ON NULL INPUT
SECURITY INVOKER
PARALLEL UNSAFE
COST 1
AS $$
BEGIN
PERFORM negozi.aggiorna_magazzino_fornitore (
NEW.fornitore,
NEW.prodotto,
NEW.quantita
);
RETURN NEW;
END;
$$;

CREATE FUNCTION negozi.aggiorna_magazzino_fornitore (_fornitore char, _prodotto
integer, _quantita integer)
RETURNS void
LANGUAGE plpgsql
VOLATILE
CALLED ON NULL INPUT
SECURITY INVOKER
```

```

PARALLEL UNSAFE
COST 1
AS $$
BEGIN
UPDATE negozi.magazzino_fornitore mf
SET quantita = mf.quantita - _quantita
WHERE mf.piva_fornitore = _fornitore
AND mf.prodotto = _prodotto
AND mf.quantita >= _quantita;

IF NOT FOUND THEN
RAISE EXCEPTION 'Stock insufficiente: fornitore %, prodotto %, richiesti %',
_fornitore, _prodotto, _quantita;
END IF;
RETURN;
END;
$$

```

aggiorna_magazzino_fornitore(): Un trigger all'inserimento dell'ordine provoca l'esecuzione della funzione trigger che chiama la funzione `aggiorna_magazzino_fornitore` che con i dati del fornitore, il prodotto e la quantità aggiorna il magazzino del fornitore sottraendo le quantità trasferita se la quantità a disposizione è sufficiente altrimenti provoca un'eccezione e non aggiorna il magazzino. L'eccezione provoca un ROLLBACK e elimina anche l'inserimento dell'ordine.

Contesto:

Trigger AFTER INSERT nella tabella `ordini_fornitori`

```

CREATE TRIGGER trg_aggiorna_ordine
AFTER UPDATE OF stato_ordine
ON negozi.ordini_fornitori
FOR EACH ROW
EXECUTE PROCEDURE negozi.trg_aggiorna_ordine();

CREATE FUNCTION negozi.trg_aggiorna_ordine ()
RETURNS trigger
LANGUAGE plpgsql
VOLATILE
CALLED ON NULL INPUT
SECURITY INVOKER
PARALLEL UNSAFE
COST 1

```

```

AS $$
BEGIN
    IF NEW.stato_ordine IS NOT DISTINCT FROM OLD.stato_ordine THEN
        RETURN NEW;
    END IF;

    PERFORM negozi.aggiorna_ordine (
        NEW.id_ordine, NEW.stato_ordine, OLD.stato_ordine);

    RETURN NEW;
END;
$$;

CREATE FUNCTION negozi.aggiorna_ordine (_id_ordine integer, _new_stato text,
_old_stato text)
RETURNS void
LANGUAGE plpgsql
VOLATILE
CALLED ON NULL INPUT
SECURITY INVOKER
PARALLEL UNSAFE
COST 1
AS $$
DECLARE
v_quantita INT := 0;
v_prodotto INT;
v_negozio INT;
v_fornitore CHAR(11);

BEGIN

    SELECT quantita, prodotto, negozio, fornitore INTO v_quantita, v_prodotto, v_negozio,
        v_fornitore
    FROM negozi.ordini_fornitori o
    WHERE o.id_ordine = _id_ordine;

    IF NOT FOUND THEN
        RETURN;
    END IF;

    IF _old_stato = 'emesso' AND _new_stato = 'consegnato' THEN
        UPDATE negozi.listino_negozio SET magazzino = magazzino + v_quantita
        WHERE negozio = v_negozio AND prodotto = v_prodotto;

```

```

RETURN;
END IF;

IF _old_stato = 'emesso' AND _new_stato = 'annullato' THEN
    UPDATE negozi.magazzino_fornitore SET quantita = quantita + v_quantita
    WHERE piva_fornitore = v_fornitore AND prodotto = v_prodotto;

    RETURN;
END IF;

RETURN;
END;
$$;

```

aggiorna_ordine(): quando un negozio effettua un ordine a un fornitore la quantità ordinata viene scalata dal magazzino fornitore ma non aumenta nel magazzino/listino del negozio perchè si assume che l'ordine non sia ancora arrivato (le specifiche del progetto chiedevano di registrare anche una data di presunto arrivo dell'ordine). I tre stati ammissibili sono "emesso", "consegnato", "annullato". All'inserimento lo stato è emesso, il negozio può aggiornare lo stato (nel nostro caso sempre il manager), da emesso a annullato o consegnato. Se viene consegnato aumenta solo la quantità del magazzino negozio se viene annullato il magazzino al magazzino fornitore viene aggiunta nuovamente la quantità dell'ordine che era stata detratta all'inserimento.

Contesto:

Trigger AFTER UPDATE su stato_ordine di un ordine.

5.3 Altre Funzioni

Queste sono funzioni chiamate via SQL call da codice PHP o all'interno del codice SQL del DB.

```

CREATE FUNCTION negozi.crea_tessera_cliente (_id_cliente integer, _id_negozio integer)
RETURNS integer
LANGUAGE plpgsql
VOLATILE
CALLED ON NULL INPUT
SECURITY INVOKER
PARALLEL UNSAFE

```

```

COST 1
AS $$
DECLARE v_id_tessera INT;
BEGIN

IF NOT EXISTS (
    SELECT 1
      FROM negozi.clienti
     WHERE id_cliente = _id_cliente
) THEN
    RETURN NULL;
END IF;

IF EXISTS (
    SELECT 1
      FROM negozi.clienti
     WHERE id_cliente = _id_cliente
       AND tessera IS NOT NULL
) THEN
    RETURN NULL;
END IF;

INSERT INTO negozi.tessere (negozio_emittente, data_richiesta, saldo_punti)
VALUES (_id_negozio, CURRENT_DATE, 0)
RETURNING id_tessera INTO v_id_tessera;

IF v_id_tessera IS NOT NULL THEN
    UPDATE negozi.clienti SET tessera = v_id_tessera
     WHERE id_cliente = _id_cliente;
    RETURN v_id_tessera;
ELSE
    RETURN NULL;
END IF;
END;
$$;

```

crea_tessera_cliente(id_cliente, id_negozio): la funzione viene chiamata quando un negozio emette una tessera cliente. Abbiamo immaginato uno scenario in cui il cliente fa una richiesta di emissione direttamente a un negozio e il negozio esegue l'emissione. Quindi il negozio emette la tessera scegliendo il cliente a cui emetterla (vede solo la lista dei clienti senza tessera) o in alternativa il cliente può richiedere direttamente la tessera

dal proprio profilo scegliendo un negozio. La tessera viene creata automaticamente con i soli *id_cliente* e *id_negozio*, quindi inserimento nella tabella *tessere* e aggiornamento del campo tessera nella tabella *clienti*.

```
CREATE FUNCTION negozi.lista_ordini_fornitore (_piva_fornitore char)
  RETURNS TABLE (nome_fornitore text, nome_prodotto text, nome_negozio text,
    quantita integer, data_ordine timestamp, stato_ordine text)
  LANGUAGE plpgsql
  VOLATILE
  CALLED ON NULL INPUT
  SECURITY INVOKER
  PARALLEL UNSAFE
  COST 1
  AS $$
BEGIN
  RETURN QUERY
  SELECT f.nome_fornitore,
         p.nome_prodotto,
         n.nome_negozio,
         of.quantita,
         of.data_ordine,
         of.stato_ordine
  FROM negozi.ordini_fornitori of
  LEFT JOIN negozi.fornitori f ON f.piva = of.fornitore
  LEFT JOIN negozi.prodotti p ON p.id_prodotto = of.prodotto
  LEFT JOIN negozi.negozio n ON n.id_negozio = of.negozio
  WHERE of.fornitore = _piva_fornitore
  ORDER BY of.data_ordine DESC, of.id_ordine DESC;
END;
$;
```

lista_ordini_fornitore(id_fornitore): la funzione permette di ricavare la lista degli ordini effettuati da negozi presso un determinato fornitore ordinati per data decrescente. La funzione prende in input la PIVA del fornitore e restituisce gli ordini effettuati dai negozi.

```
CREATE FUNCTION negozi.miglior_fornitore (_prodotto integer)
  RETURNS char
  LANGUAGE plpgsql
  VOLATILE
  CALLED ON NULL INPUT
  SECURITY INVOKER
```



```

PARALLEL UNSAFE
COST 1
AS $$
DECLARE
v_fornitore CHAR(11);
BEGIN
    SELECT piva_fornitore INTO v_fornitore
    FROM negozi.magazzino_fornitore mf
    JOIN negozi.fornitori f ON mf.piva_fornitore = f.piva
    WHERE mf.prodotto = _prodotto AND
           mf.quantita = _quantita AND
           f.attivo = true
    ORDER BY mf.prezzo ASC LIMIT 1;
    RETURN v_fornitore;
END;
$$;

```

miglior_fornitore(prodotto, quantità): quando un negozio ordina un prodotto da un fornitore la richiesta è che venga selezionato automaticamente il miglior fornitore che abbia in magazzino una quantità maggiore o uguale a quella richiesta in base al prezzo per il prodotto richiesto. La scelta deve essere automatica e la funzione ritorna il primo risultato di una query che ordina il prodotto per tutti i fornitori in ordine crescente di prezzo.

```

CREATE FUNCTION negozi.tessere_negozi (_id_negozi integer)
RETURNS TABLE (id_cliente integer, nome text, cognome text, numero_tessera integer,
data_richiesta date, saldo_punti integer)
LANGUAGE plpgsql
VOLATILE
CALLED ON NULL INPUT
SECURITY INVOKER
PARALLEL UNSAFE
COST 1
AS $$

BEGIN
    RETURN QUERY
    SELECT c.id_cliente AS id_cliente,
           c.nome,
           c.cognome,
           c.tessera AS numero_tessera,
           t.data_richiesta,
           COALESCE(t.saldo_punti,0) AS saldo_punti

```

```
FROM negozi.clienti c
  JOIN negozi.tessere t ON t.id_tessera = c.tessera
WHERE t.negozio_emittente = _id_negozio;
END;
$$;
```

tessere_negozio(id_negozio): fornisce una lista di tessere emesse da un determinato negozio.

6. Casi d'uso (sintesi)

Il LOGIN è unico per clienti e manager, a seconda del ruolo dell'utente che si collega verranno attivate le funzionalità cliente o manager.

Immagine:

1_login.png

6.1 Funzionalità Cliente

L'idea era se creare una dashboard vuota e selezionare i negozi da cui comprare o semplicemente fornire subito una lista di prodotti disponibili. La richiesta di avere un sistema di fatturazione unica mi ha suggerito l'idea di usare la dashboard cliente come una vetrina delle vetrine. Solo dopo aver scelto il prodotto il cliente seleziona il negozio da cui comprare in base al prezzo migliore. Anche se commercialmente avrebbe poco senso (concorrenza tra negozi della stessa catena) questa mi sembra la scelta che meglio rispecchia le specifiche.

6.1.1 Dashboard

Immagini:

2_dashboad_prodotti.png

6.1.2 Carrello

Immagini:

3_carrello_vuoto.png

4_carrello_pieno.png

5_selezione_sconto.png

6_sconto_meno_di_100.png

6b_acquisto_completato.png

6.1.2 Pagina Profilo

Immagini:

7_profilo_con_fattura.png

7a_profilo_con_fattura.png

6.2 Funzionalità Manager

Al login il manager entra nella propria dashboard dove gestisce clienti, negozi, prodotti e fornitori. Esistono anche dei pulsanti di scelta rapida per le azioni più usate.

Immagine:

8_dashboard_manager.png

6.2.1 Gestione Clienti

Immagini:

9_lista_clienti.png

9b_lista_clienti_300_punti.png

10_nuovo_cliente.png

La stessa pagina per la creazione di un cliente viene usata come maschera per la modifica di un cliente (comilata con i dati attuali del cliente selezionato). Un nuovo cliente viene creato senza tessera.

6.2.2 Gestione Negozi

Immagini:

11_lista_negozi.png

12_nuovo_negozio.png

La stessa maschera usata per la modifica negozio.

Tessere e Listino sono selezionabili e aprono la vista delle tessere emesse dal negozio.

13_lista_tessere_negozio.png

14_nuova_tessera.png

Se esistono dei clienti senza tessera un negozio può assegnare una propria tessera al cliente.

15_listino_negozio.png

Da questa pagina si possono modificare i listini, quantità, prezzi, cancellare prodotti in listino o ordinare prodotti non in listino o aumentare la quantità di prodotti già in listino.

16_aggiungi_prodotto_listino.png

16a_aggiungi_prodotto_listino.png

Se un prodotto non è in listino si chiede di inserire anche il prezzo (se un prodotto è già in listino si specifica solo la quantità), poi verrà ordinato dal miglior fornitore che ha in magazzino almeno la quantità richiesta.

17_inserisci_ordine_fornitore.png

17a_conferma_ordine.png

17b annulla_ordine.png

6.2.3 Gestione Prodotti

Immagini:

18_lista_prodotti.png

19_aggiungi_prodotto.png

La maschera per la modifica dei dati di un prodotto è la stessa della maschera per l'aggiunta.

20_nuovo_prodotto.png

6.2.4 Gestione Fornitori

21_lista_fornitori.png

22_nuovo_fornitore.png

La maschera di modifica dati fornitore è la stessa della maschera per la aggiunta di un fornitore.

Un fornitore viene creato con magazzino vuoto. La pagina dashboard fornitore mostra per ogni fornitore la quantità totale del magazzino (link al magazzino) e il link alla lista ordini effettuati a questo fornitore.

23_lista_magazzino.png

Dal magazzino si può selezionare un prodotto da aggiungere (se il prodotto non esiste in magazzino verrà richiesto anche il prezzo) insieme alla quantità.

24_aggiungi_prodotto_magazzino.png

Non sono previste funzioni che svuotino prodotti dal magazzino o da listini di negozi non attivi perchè solo quelli dei negozi/fornitori attivi vengono mostrati come disponibili.

25_lista_ordini_stato.png

I fornitori possono accedere dalla dashboard alla lista/stato degli ordini.

Schema Logico (relazionale)

negozi_db

auth

utenti(id_utente, ruolo, email, password, attivo, creato, ultimo_accesso*)

ruolo(id_ruolo, nome, descrizione*)

FK

utenti.ruolo → ruolo.id_ruolo

negozi

clienti(id_cliente, cf, nome, cognome*, utente, telefono*, tessera)

tessere(id_tessera, negozio_emittente, data_richiesta, saldo_punti)

negozi(id_negozio, nome_negozio, responsabile, indirizzo, attivo)

fornitori(piva, nome_fornitore, indirizzo, email, telefono, attivo)

prodotti(id_prodotto, nome_prodotto, descrizione*, immagine_url*)

magazzino_fornitore(piva_fornitore, prodotto, quantita, prezzo)

ordini_fornitori(id_ordine, fornitore, negozio, prodotto, quantita, data_ordine, data_consegna, stato_ordine)

storico_tessere(id_storico, codice_tessera, cliente, saldo_punti, negozio_emittente, data_emissione)

fatture(id_fattura, cliente, data_fattura, sconto_percentuale, totale_pagato)

dettagli_fattura(fattura, prodotto, quantita, prezzo_unita)

livelli_sconto(sconto_percentuale, punti_richiesti)

listino_negozio(negozio, prodotto, prezzo_listino, magazzino)

orari(negozio, dow, iod, apertura, chiusura)

FK

clienti.utente → **auth**.utenti.id_utente

clienti.tessera → tessere.id_tessera

tessere.negozio_emittente → negozi.id_negozio

magazzino_fornitore.piva_fornitore → fornitori.piva

magazzino_fornitore.prodotto → prodotti.id_prodotto

ordini_fornitori.fornitore → fornitori.piva

ordini_fornitori.negozio → negozi.id_negozio

ordini_fornitori.prodotto → prodotti.id_prodotto

fatture.cliente → clienti.id_cliente

dettagli_fattura.fattura → fatture.id_fattura

dettagli_fattura.prodotto → prodotti.id_prodotto

listino_negozio.negozio → negozi.id_negozio

listino_negozio.prodotto → prodotti.id_prodotto